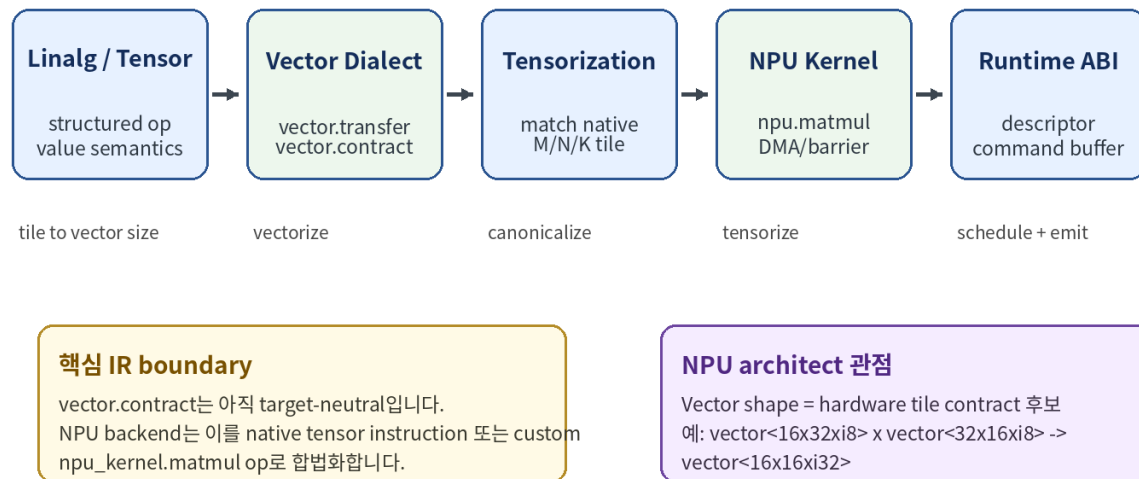


MLIR for AI / NPU Compiler

Lecture 14 - Vector Dialect and Tensorization

Topic: vector.contract -> NPU matmul mapping | Duration: 90-120 min | Target: AI/NPU compiler engineers

Lecture 14: Vector Dialect and NPU Tensorization Pipeline



1. Learning Objectives

- MLIR Vector Dialect 가 tensor/linalg 와 memref/LLVM/NPU backend 사이에서 맡는 역할을 설명한다.
- vector type, vector.transfer_read/write, vector.contract 의 핵심 필드를 읽고 NPU tile compute 와 연결한다.
- vector.contract 를 native NPU tensor instruction 또는 custom npu_kernel.matmul op 로 tensorization 하는 legality 조건을 정의한다.
- mask, in_bounds, padding, transpose, accumulator type 이 correctness 와 performance 에 미치는 영향을 audit 한다.
- FileCheck 기반으로 tensorization positive/negative regression test 를 작성한다.

2. Prerequisites and Position in Course

- 7 강 Linalg-on-Tensors: structured op, indexing_maps, iterator_types 개념을 복습한다.
- 8 강 SCF/Affine: tile loop 와 DMA 순서가 vector tile 을 둘러싼 control skeleton 이 됨을 이해한다.
- 12 강 Bufferization, 13 강 MemRef: tile buffer, memory space, layout, descriptor 가 vector transfer 와 연결된다.
- 14 강의 핵심 산출물은 vector.contract -> npu_kernel.matmul tensorization mapping table 이다.

3. Vector Dialect Big Picture

MLIR Vector Dialect 는 hardware-independent vector abstraction 이다. 특히 n-D vector 는 단순 SIMD lane 묶음이 아니라 tile shape, contraction structure, transfer permutation, mask 정보를 IR 에 남겨 target lowering 이 활용할 수 있게 만든다.

Layer	Representative IR	Compiler Decision	NPU Meaning
Tensor/Linalg	linalg.matmul ins/outs tensor	fusion, tiling, layout propagation	아직 physical tile 아님
Vector	vector.transfer_read, vector.contract	native tile match, vector unroll, mask policy	NPU tensor op 후보
MemRef/SCF/Affine	memref.subview, scf.for	buffer placement, DMA, loop order	SRAM/DRAM 이동과 schedule
NPU Dialect	npu_kernel.matmul, npu.dma	target legal op + command sequence	hardware contract

4. Vector Type Anatomy

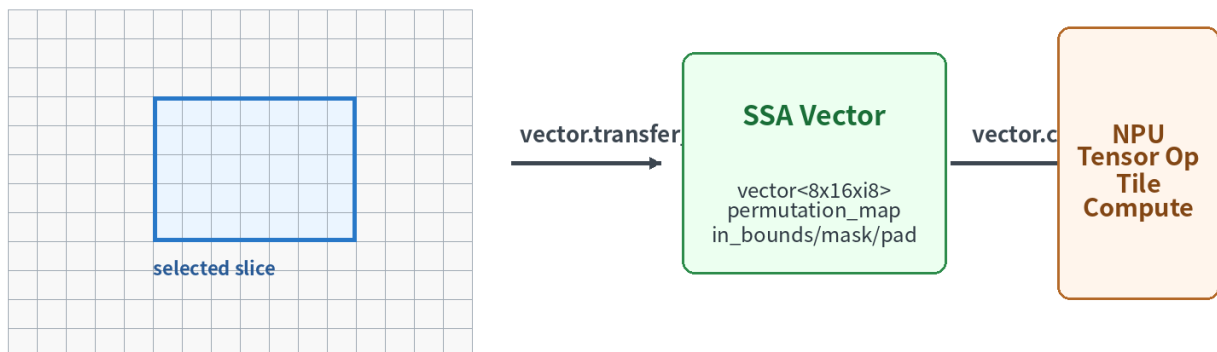
예를 들어 `vector<16x32xi8>`은 2-D vector 이며, shape `[16,32]`와 element type `i8`을 갖는다. NPU compiler 에서는 이 shape 를 M/K tile 로 해석할 수 있지만, 그 의미는 `vector.contract` 의 `indexing_maps` 와 `iterator_types` 를 함께 봐야 확정 된다.

```
// Examples of vector values that may correspond to NPU tiles.
%a = vector.transfer_read %A[%m, %k], %c0_i8
    {in_bounds = [true, true]} : memref<?x?xi8>, vector<16x32xi8>
%b = vector.transfer_read %B[%k, %n], %c0_i8
    {in_bounds = [true, true]} : memref<?x?xi8>, vector<32x16xi8>
%acc = arith.constant dense<0> : vector<16x16xi32>
```

5. vector.transfer_read / vector.transfer_write

vector.transfer_read / transfer_write as Tile Movement

MemRef / SRAM Tile Buffer



write-back path `vector.transfer_write %Ctile, %C[%m,%n] : vector<16x16xi32>, memref<?x?xi32, #dram>`

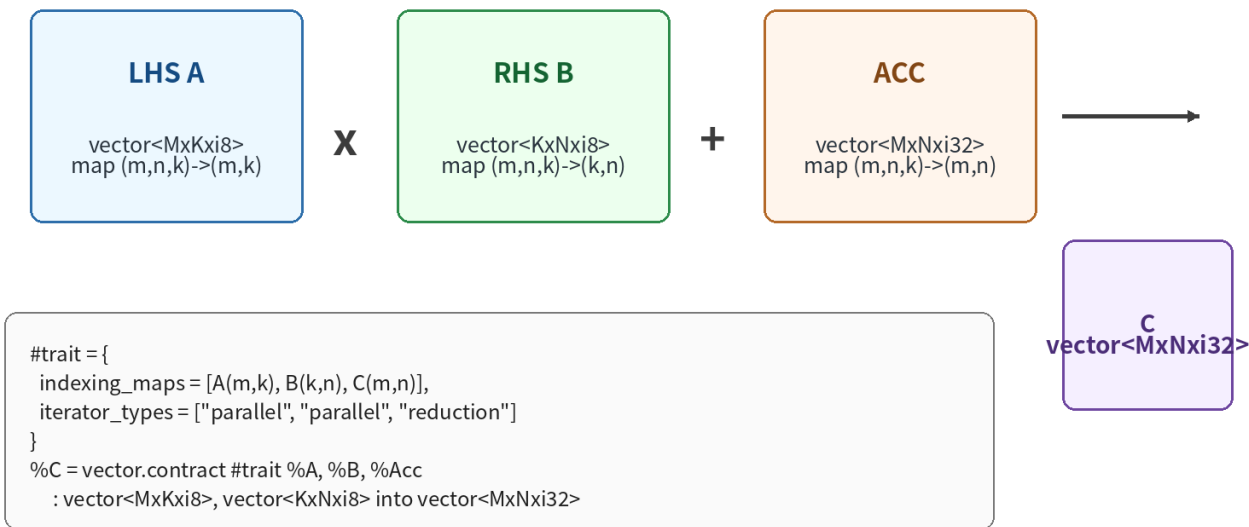
중요: transfer op는 load/store보다 큰 tile-level abstraction입니다. boundary mask, padding, permutation을 audit해야 합니다.

transfer_read/write 는 scalar load/store 보다 높은 abstraction 이다. slice 시작점, result vector shape, permutation_map, padding, mask/in_bounds 를 통해 tile 이동의 의미를 보존한다.

Field	Meaning	NPU Audit Question
base + indices	memref/tensor slice 시작점	DMA source/destination address 가 tile-aligned 인가?
vector type	읽고 쓸 super-vector shape	native M/N/K tile 과 맞는가?
permutation_map	memory dimension -> vector dimension mapping	layout transpose/repack 이 필요한가?
padding	out-of-bounds element value	edge tile 에서 zero padding 이 정확한가?
mask/in_bounds	boundary 또는 predication 정보	NPU predicated matmul 을 지원하는가?

6. vector.contract Semantics

vector.contract Anatomy: MatMul Form



NPU tensorization은 이 contract가 native tile shape, dtype, layout, accumulator 정책과 일치하는지 검증합니다.

MatMul 형태의 vector.contract 는 lhs/rhs 의 contracting dimension 을 따라 product 를 reduce 하고 accumulator 에 combine 한다. NPU matmul tensorization 의 핵심은 이 contraction trait 가 hardware tensor instruction 의 semantics 와 정확히 일치하는지 증명하는 것이다.

```
#matmul_trait = {
  indexing_maps = [
    affine_map<(m, n, k) -> (m, k)>,
    affine_map<(m, n, k) -> (k, n)>,
    affine_map<(m, n, k) -> (m, n)>
  ],
  iterator_types = ["parallel", "parallel", "reduction"]
}
%c = vector.contract #matmul_trait %a, %b, %acc
: vector<16x32xi8>, vector<32x16xi8> into vector<16x16xi32>
```

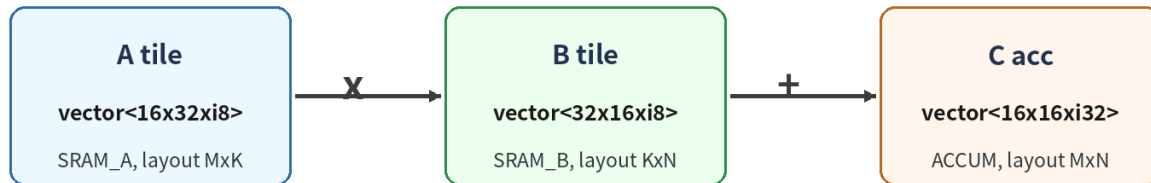
7. Vectorization vs Tensorization

Term	Goal	Typical Output	NPU Compiler Interpretation
Vectorization	structured op 를 vector IR 로 표현	vector.transfer + vector.contract	target-neutral tile compute IR
Vector lowering	high-rank vector 를 lower-rank/native vector/LLVM 등으로 낮춤	vector.fma, scf loops, LLVM vector	CPU/GPU/SIMD path 에 적합
Tensorization	vector.contract 를 hardware tensor op 로 치환	npu_kernel.matmul / mma / dot	NPU native compute path
Fallback lowering	지원 안 되는 vector op 를 generic lowering	SCF/arith/memref 또는 CPU fallback	성능 저하 가능성 큼

8. Native NPU Tile Shape and Legality

Mapping vector.contract to Native NPU MatMul

Example native tile: M=16, N=16, K=32, A/B=int8, Acc=int32, epilogue=optional requant/activation



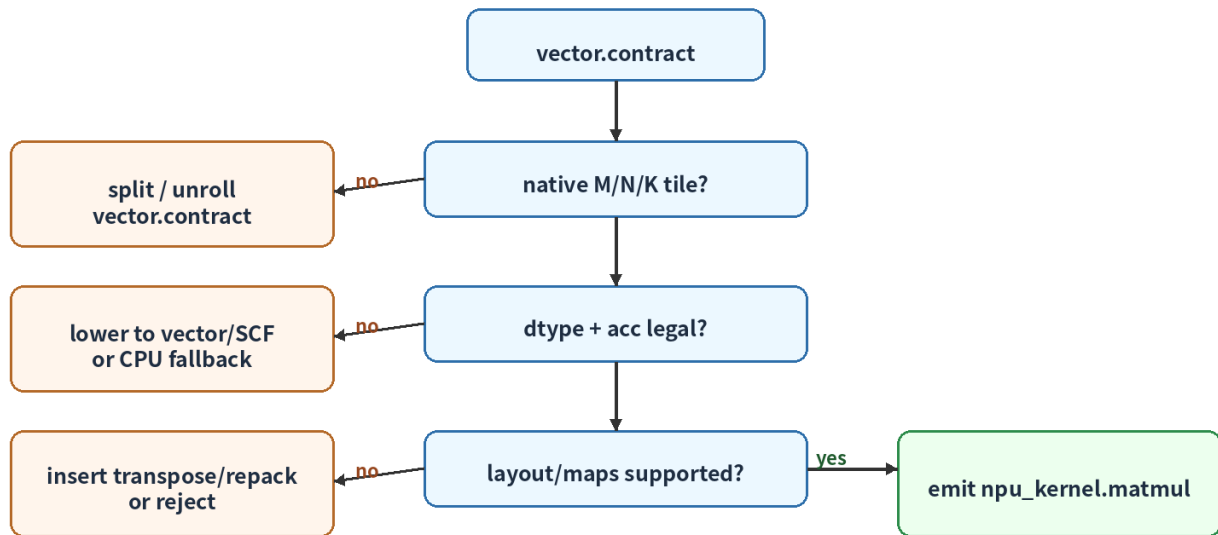
```
npu_kernel.matmul {m=16, n=16, k=32, lhs_layout="mk",  
rhs_layout="kn", acc_type="i32"}
```

예시 target 은 M16N16K32 INT8 matmul tile 을 native 로 지원한다고 가정한다. 실제 compiler 에서는 target description 또는 cost model 이 이 정보를 제공해야 한다.

Check	Example Rule	Failure Action
Shape	lhs=16x32, rhs=32x16, acc=16x16	split/unroll 또는 reject
Iterator types	[parallel, parallel, reduction]	not a matmul contract -> fallback
Indexing maps	A(m,k), B(k,n), C(m,n)	transpose/repack 또는 reject
DType	i8 x i8 -> i32, bf16 x bf16 -> f32	unsupported dtype diagnostic
Accumulator	acc/result type matches hardware accumulation	insert cast/requant or reject
Mask/remainder	full tile only or supported predication	peel edge tile / masked fallback
Layout/memory	operands in SRAM, alignment satisfied	insert DMA/repack/copy

9. Tensorization Decision Tree

NPU Tensorization Decision Tree



Rule of thumb: tensorize only when the IR expresses the hardware contract explicitly enough to prove correctness.

NPU tensorization 은 “vector.contract 이름이 보인다”가 아니라 “hardware contract 를 증명할 수 있다”가 기준이다. shape, dtype, accumulator, layout, mask, memory-space 를 모두 확인해야 한다.

10. Pseudo Lowering Pattern

```
// Before: target-neutral vector IR.
%c = vector.contract #matmul_trait %a, %b, %acc
      : vector<16x32xi8>, vector<32x16xi8> into vector<16x16xi32>

// After: target-specific NPU kernel op.
%c2 = npu_kernel.matmul %a, %b, %acc
      {m = 16 : i64, n = 16 : i64, k = 32 : i64,
       lhs_layout = "mk", rhs_layout = "kn", acc_type = "i32"}
      : vector<16x32xi8>, vector<32x16xi8>, vector<16x16xi32>
      -> vector<16x16xi32>
```

11. Recommended Pass Pipeline

```
# Conceptual pass pipeline for Lecture 14.
mlir-opt input.mlir --pass-pipeline='builtin.module(
  func.func(
    canonicalize,cse,
    linalg-generalize-named-ops,
    linalg-fuse-elementwise-ops,
    linalg-tile-to-vector-size,
    linalg-vectorize,
    canonicalize,cse,
    npu-vector-contract-tensorize,
    npu-vector-transfer-legalize,
    npu-kernel-schedule
  ))'
```

12. FileCheck Strategy

positive test 는 `vector.contract` 가 사라지고 `npu_kernel.matmul` 이 생성되는지 확인한다. negative test 는 `unsupported shape/mask/dtype` 에 대해 `tensorization` 이 발생하지 않거나 `diagnostic` 을 내는지 확인한다.

```
// CHECK-LABEL: func.func @m16n16k32_i8
// CHECK: npu_kernel.matmul
// CHECK-SAME: m = 16
// CHECK-SAME: n = 16
// CHECK-SAME: k = 32
// CHECK-NOT: vector.contract

// NEGATIVE-LABEL: func.func @unsupported_k48
// NEGATIVE-NOT: npu_kernel.matmul
// NEGATIVE: vector.contract
```

13. NPU Hardware Architect Notes

- Vector shape 는 compiler-only abstraction 일 수도 있고, hardware tile ISA 와 1:1 대응될 수도 있다. 두 경우를 명확히 분리한다.
- M/N/K tile 을 고정하면 compiler 는 쉽지만 dynamic shape 와 edge tile 처리 비용이 증가한다.
- K tile 이 accumulator reuse 와 SRAM bandwidth 를 좌우한다. INT8 에서는 K granularity 가 dot-product datapath width 와 맞아야 한다.
- `vector.transfer_read` 의 `permutation_map` 을 무시하면 layout bug 가 생긴다. 특히 NHWC/HWCF, blocked layout, transposed weight 에 주의한다.
- mask/predication 지원이 없는 NPU 에서는 edge tile 을 loop peeling 또는 padded copy 로 처리해야 한다.

14. In-Class Exercises

Exercise	Task	Output
A	<code>vector.transfer_read</code> 의 <code>base/indices/padding/in_bounds</code> 를 annotation	tile movement table
B	MatMul <code>vector.contract</code> 의 <code>indexing_maps/iterator_types</code> 해석	M/N/K mapping table
C	M16N16K32 target legality check 수행	tensorization decision
D	<code>vector.contract</code> 를 <code>npu_kernel.matmul</code> pseudo IR 로 rewrite	before/after MLIR
E	FileCheck positive/negative test 작성	regression test file

15. Homework

- BF16 MatMul `vector.contract` 예제를 작성하고, accumulator f32 policy 를 명시하라.
- M32N8K64 native tile 을 가정하고 기존 M16N16K32 체크리스트를 수정하라.
- edge tile 이 M=13, N=16, K=32 일 때 padding vs predication vs fallback 중 하나를 선택하고 이유를 설명하라.
- `npu-vector-contract-tensorize` pass 의 negative diagnostics 3 개를 설계하라.

16. References

- MLIR Vector Dialect: <https://mlir.llvm.org/docs/Dialects/Vector/>
- MLIR Passes: convert-vector-to-llvm / convert-vector-to-scf / architecture-specific vector lowering: <https://mlir.llvm.org/docs/Passes/>
- MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>
- MLIR Transform Dialect: <https://mlir.llvm.org/docs/Dialects/Transform/>